1. Problem Statement & Objectives

1.1 Problem Statement

Traditional healthcare facilities often struggle with fragmented administrative workflows, reliance on paper-based record-keeping, and inefficient patient-scheduling pipelines. These operational bottlenecks lead to several core issues:

- **Inaccurate & Lost Records:** Paper charts or unstandardized digital logs risk data misplacement, transcription errors, and fragmented patient history across departments.
- **Scheduling Inefficiencies:** Without dynamic slot reservation, manual appointment tracking leads to double-booking, long patient waiting times, and unoptimized doctor schedules.
- **Disconnected Financial & Clinical Operations:** Lacking integration between booking, payment processing, and medical record updates increases administrative overhead and entry mistakes.
- **Access Control & Privacy Risks:** Absence of standardized Role-Based Access Control (RBAC) increases the risk of unauthorized access to sensitive patient health data.

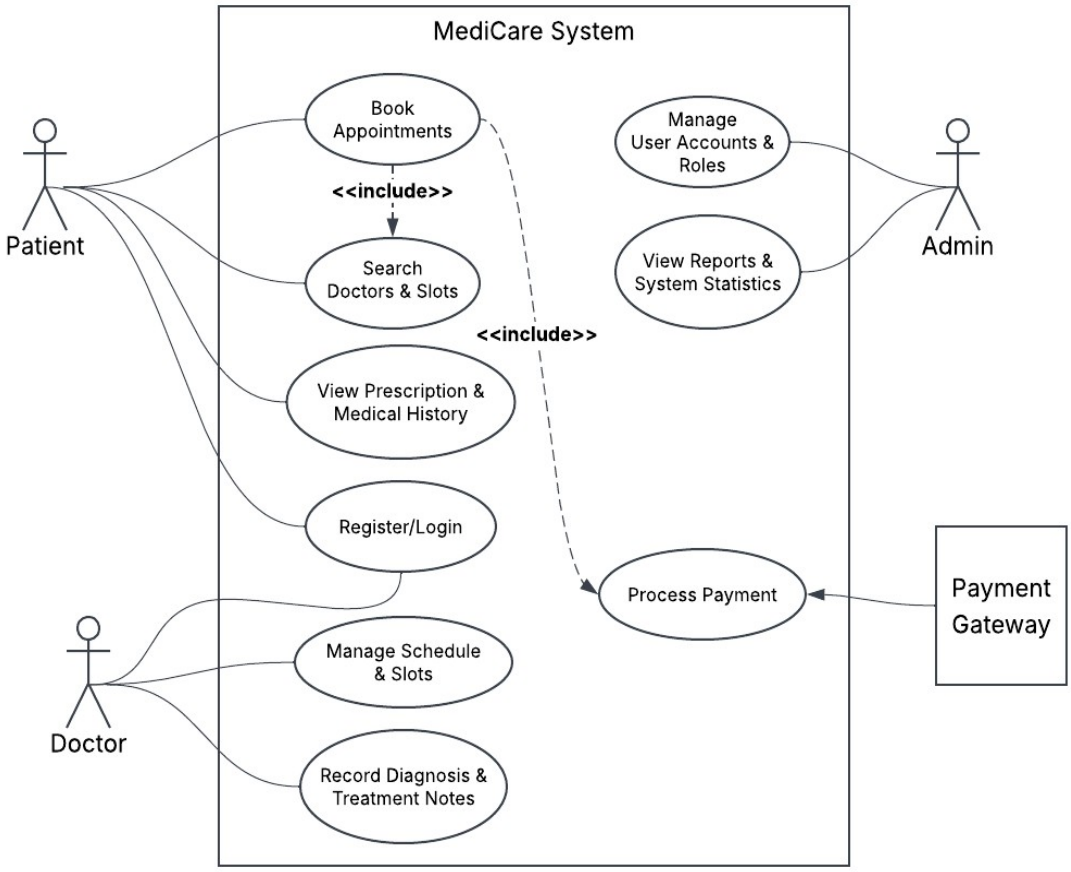
1.2 Project Goals & Objectives

The primary objective of the project is to design and develop a web-based Clinic Management Application using an **N-Tier architecture** to digitize operations, secure health records, automate scheduling, and process online transactions.

Specific system objectives include:

1. **Centralize Patient Health Records:** Provide a secure repository for medical histories, diagnoses, symptoms, and treatment notes linked directly to appointments.
2. **Automate Schedule & Slot Management:** Allow doctors to configure weekly availability and enable real-time slot reservation for patients.
3. **Integrate Secure Payment Processing:** Handle online appointment payments via a dedicated Payment Gateway prior to booking confirmation.
4. **Enforce Role-Based Access Control (RBAC):** Restrict system features across four explicit roles: Patient, Doctor, Admin, and Payment Gateway.
5. **Maintain Decoupled Architecture:** Implement design patterns like **Repository**, **Unit of Work**, and **Dependency Injection** to separate data access operations from core application logic.

2. Use Case Diagram & System Actors



2.1 System Actors

Based on the system boundaries established in the DFD Context Diagram:

- **Patient:** Registers/logs in, searches doctors by specialty, views available time slots, reserves appointments, completes payment, and views medical histories and digital prescriptions.
- **Doctor:** Manages profile and weekly working hours/slots, views assigned schedules, and logs patient symptoms, diagnoses, and treatment notes.
- **Admin:** Manages user and doctor account credentials, system roles, and generates operational reports and system statistics.
- **Payment Gateway (External Entity):** Receives payment requests during booking and returns transaction authorization statuses.

2.2 Core Use Case Descriptions

Use Case 1: Search Doctors & Book Appointment

- **Primary Actor:** Patient
- **Preconditions:** Patient is authenticated; Doctor has active working hours configured in the system.
- **Main Success Flow:**
    1. Patient enters search criteria (Specialty, preferred date).
    2. System retrieves matching doctors and available time slots from the schedules data store.
    3. Patient selects a doctor and slot, then provides payment details.
    4. System routes payment details to the **Payment Gateway** process.
    5. Upon receiving payment confirmation, the system creates an appointment record with status Scheduled and issues a booking confirmation to the patient.
- **Alternative / Exception Flows:**
    - *Payment Failed:* If the payment gateway returns a failure status, the system cancels the slot reservation and prompts the patient to retry.

Use Case 2: Record Diagnosis & Medical History

- **Primary Actor:** Doctor
- **Preconditions:** Doctor is authenticated and viewing an assigned patient appointment.
- **Main Success Flow:**
    1. Doctor opens the patient appointment details.
    2. Doctor inputs clinical details (Symptoms, Diagnosis, TreatmentNotes).
    3. Doctor submits the record.
    4. System persists the entry into the Medical records data store linked to the specific AppointmentId, PatientId, and DoctorId.
    5. Patient's updated medical history becomes immediately accessible on their dashboard.

3. Functional & Non-Functional Requirements

3.1 Functional Requirements (FR)

- **Authentication & Accounts:** The system shall register and authenticate Patients, Doctors, and Admins, storing credentials in the Users data store.

- **Schedule Management:** The system shall allow Doctors to define weekly availability by setting DayOfWeek, StartTime, and EndTime in the Working hours module.

- **Doctor Search:** The system shall allow Patients to search and filter Doctors by Specialization.

- **Slot Reservation & Payment:** The system shall temporarily lock selected time slots, forward billing details to the Payment Gateway, and confirm appointments upon verified payment.

- **Medical Records:** The system shall allow Doctors to enter diagnoses, symptoms, and treatment notes tied to completed appointments.

- **Admin Management & Reporting:** The system shall allow Admins to manage user/doctor profiles and view system statistics and reports.

3.2 Non-Functional Requirements (NFR)

- **Security:** All user passwords must be hashed using industry-standard cryptography before storage in the database (Users entity). Communications with the Payment Gateway must occur over encrypted HTTPS/TLS channels.

- **Data Integrity & Normalization:** Database relationships must maintain strict foreign key referential integrity across Users, Doctors, Patients, Appointments, and Medical records.

- **Performance & Availability:** Slot availability queries and doctor searches shall execute in under 2 seconds during peak traffic.

4. Software & Database Architecture

4.1 High-Level Architecture (N-Tier Style)

The system is built on an **N-Tier Architectural Pattern** to separate concerns:

1. **Presentation Layer (Frontend):** Renders user interfaces for Patients, Doctors, and Admins to interact with forms, view schedules, and trigger bookings.

2. **Application / Business Logic Layer:** Contains domain logic for schedule validation, appointment slot locking, payment verification routing, and RBAC rules.

3. **Data Access Layer (DAL):** Implements the **Repository Pattern** and **Unit of Work Pattern** with **Dependency Injection** to execute decoupled CRUD operations against the relational database.

4. **Database Layer (SQL Server):** Relational storage organized according to the normalized Entity-Relationship Diagram.

4.2 Database Schema & ERD Mapping

The database design directly mirrors the domain entities:

- **Users (**Users**):** Base authentication entity linked to specific Doctors and Patients records via UserId foreign keys.

- **Doctors (**Doctors**) & Specialization (**Specialization**):** Doctors store profile data (Bio, ConsultationFee, ProfileImageUrl) and link to a Specialization (SpecializationId).

- **Working Hours (**Working hours**):** Foreign-keyed to Doctors (DoctorId), mapping available slots using DayOfWeek, StartTime, and EndTime.

- **Appointments (**Appointments**):** Core relational table linking DoctorId and PatientId along with AppointmentDate, StartTime, Status, and PatientNotes.

- **Medical Records (**Medical records**):** Linked directly to PatientId, DoctorId, and AppointmentId, capturing Symptoms, Diagnosis, and TreatmentNotes.